

Agents and Workflows

Domain 1 of the CCDV-F blueprint — 14.7% of the exam.

Sources: [Agent SDK: how the agent loop works](https://code.claude.com/docs/en/agent-sdk/agent-loop) (https://code.claude.com/docs/en/agent-sdk/agent-loop), [Agent SDK: subagents](https://code.claude.com/docs/en/agent-sdk/subagents) (https://code.claude.com/docs/en/agent-sdk/subagents), [Agent SDK: hooks](https://code.claude.com/docs/en/agent-sdk/hooks) (https://code.claude.com/docs/en/agent-sdk/hooks), [Building effective AI agents](https://www.anthropic.com/engineering/building-effective-agents) (https://www.anthropic.com/engineering/building-effective-agents) (Anthropic engineering blog), [Hosting the Agent SDK](https://code.claude.com/docs/en/agent-sdk/hosting) (https://code.claude.com/docs/en/agent-sdk/hosting), [Self-hosted sandboxes](https://platform.claude.com/docs/en/managed-agents/self-hosted-sandboxes) (https://platform.claude.com/docs/en/managed-agents/self-hosted-sandboxes).

Agent Architecture (4.5%)

Workflow vs. agent — the decision that gates everything else

Anthropic draws a specific line between the two:

- **Workflows:** LLMs and tools follow a **predefined code path** — the orchestration logic lives in your code, not the model's judgment.
- **Agents:** the LLM **dynamically directs its own process and tool use**, maintaining control over how it accomplishes a task.

Decision rule from Anthropic's own engineering guidance: if you can map out the exact steps a task requires, build a **workflow** — you get predictability and consistency. If you can't predict the steps in advance, build an **agent** — you trade predictability for flexibility. If you're not sure, start with an agent and extract deterministic workflow patterns as they emerge from real usage, rather than guessing at a workflow structure up front.

Choose a workflow when...	Choose an agent when...
You can enumerate the exact steps in code	You can specify the goal and tools but not the exact path
Error cost is real and step-level guardrails matter	The path through the work can't be enumerated in advance
Standard-tooling observability is required	Non-determinism is acceptable, actions are bounded by the registered toolset
Inputs are well-constrained to a known set	Inputs vary unpredictably in content and structure
Every execution follows the same sequence	The task needs creative sequencing of available tools

Agents suit open-ended problems requiring many autonomous steps and heavy tool use, but they demand more testing, tighter guardrails, and an explicit cost-benefit case against a simpler workflow — an agent is not automatically the better default just because it's more flexible. **The progression that keeps this honest: start with the simplest pattern that solves the problem (a single API call), then a workflow, then an agent — move up only when the simpler pattern can't handle the variability the task actually requires.** Choosing the wrong pattern at the start is called out as the single most critical agent-development mistake: an agent where a workflow would do adds behavioral complexity with no capability gain (and trades standard operational logging for transcript-level observability tooling); a workflow where an agent is needed produces a system that breaks the moment user input deviates from the predetermined path.

The five named workflow patterns

These come from Anthropic's "Building Effective AI Agents" engineering post and are the standard vocabulary the exam draws on:

Pattern	What it does	Best for
Prompt chaining	Decomposes a task into sequential LLM calls, each processing the previous step's output	Fixed subtasks where decomposition improves accuracy through focused attention per step
Routing	Classifies the input, then sends it to a specialized handler (different prompt/tool/model per category)	Distinct input categories that genuinely need different handling
Parallelization	Runs subtasks simultaneously — either <i>sectioning</i> (independent subtasks) or <i>voting</i> (multiple attempts on the same task)	Speed gains (sectioning) or confidence through diverse perspectives (voting)
Orchestrator-workers	A central LLM dynamically breaks a task into subtasks it <i>can't</i> predict in advance and delegates each to a worker	Complex, variable problems where the subtask structure depends on the specific input
Evaluator-optimizer	One LLM generates a response, a second evaluates it and gives feedback for iterative refinement	Tasks with clear evaluation criteria where refinement demonstrably improves output

Orchestrator-workers is the pattern most exam questions about "agent architecture" are really testing — it's the bridge between pure workflow and pure agent: the orchestrator's *decomposition* is dynamic (agent-like) while each worker's *task* is scoped and bounded (workflow-like).

The concrete cost of orchestrator-workers: treat it as a hiring decision

Fan-out is not free. In an orchestrator-worker run, a lead agent decomposes the task and dispatches subtasks to several subagents that each run in **their own context window** and spend their own tokens; the lead then compiles the results:

```

async def orchestrate(task):
    plan = await lead.plan(task)                                # lead agent decomposes
    results = await gather(*[worker.run(s) for s in plan.subtasks]) # subagents run in parallel
    return await lead.synthesize(results)                       # lead compiles the answer

```

Anthropic's own multi-agent research system (Opus as lead, Sonnet subagents) reported a substantial quality improvement over a single-agent baseline on broad research tasks — at roughly **15x the token cost of a normal single-agent chat interaction**, because five contexts (a lead plus four workers) plus a synthesis pass each carry their own input and output tokens. The multiplier holds specifically because the work **genuinely decomposes into independent parts** explored in parallel (research across separate sources). On **tightly-coupled work where each step depends on the last — coding is the standard counterexample — the pattern is less effective**, because subagents mostly end up waiting on each other and you pay the fan-out cost without getting the parallel benefit.

The framing that keeps this honest: it's a hiring decision. Five researchers finish a broad survey faster than one, but you pay five salaries — you only "hire a team" when the work genuinely splits into parts that don't depend on each other. Real incident pattern: a developer fanned a slow, tightly-coupled task out across parallel subagents expecting a proportional speedup; latency dropped a little, the bill tripled, and answer quality barely moved — because the task's steps depended on each other and the subagents spent most of their time waiting rather than exploring in parallel. Moving the task back to a single agent with the same context dropped the bill back down with no quality loss. **Reach for orchestrator-workers only when an eval or the task's own structure confirms genuine parallel decomposability — not by default whenever a task "feels slow."** Two mitigations when you do use it: use a more capable model only for the lead and cheaper models for workers (you're not paying top-tier rates across every parallel context), and note the multiplier compounds badly on a misbehaving run — a runaway subagent or an oversized tool result can push a fan-out well past its expected multiplier before the request even completes.

Manager/supervisor hierarchies

The **supervisor (leader-worker) pattern** is the most common and best-supported multi-agent structure: a central supervisor receives the request, maintains a list of worker agents, delegates tasks, and reviews/aggregates their output — the supervisor itself typically doesn't execute the work directly. This maps directly onto how Claude Code and the Agent SDK structure subagent delegation: a primary agent instance (the "manager") coordinates specialist subagents, each with isolated context and a narrow role. A **hierarchical** variant extends this to multiple levels — a top-level manager delegates to mid-level supervisors, who delegate to leaf-level workers — useful when a single supervisor would otherwise need to track too many direct workers at once.

The role of subagents in improving task execution

Subagents improve execution along three axes, independent of which architecture pattern you're using:

1. **Context isolation** — a subagent's intermediate tool calls and reasoning never pollute the parent's context; only the final summary returns.
2. **Parallelization** — independent subtasks finish in the time of the slowest one, not the sum of all of them.
3. **Specialization** — a subagent's system prompt can carry narrow, deep domain knowledge that would be noise in the main agent's instructions, and its tool access can be scoped down to reduce the blast radius of a mistake.

Agent Construction with Claude (5.3%)

The Claude Agent SDK and the agent loop

The Agent SDK (Python: `claude-agent-sdk`, TypeScript: `@anthropic-ai/claude-agent-sdk`; renamed from the Claude Code SDK in early 2026) embeds the same autonomous execution loop that powers Claude Code, programmable outside the CLI. Every session follows the same cycle:

1. **Receive prompt** — system prompt, tool definitions, and conversation history go in; the SDK yields a `SystemMessage ( subtype: "init" )` with session metadata.
2. **Evaluate and respond** — Claude responds with text, tool-call requests, or both (`AssistantMessage`).
3. **Execute tools** — the SDK runs each requested tool and feeds results back to Claude (`UserMessage` with tool results). [Hooks](#) can intercept, modify, or block a tool call before it runs.
4. **Repeat** — steps 2–3 cycle until Claude produces a response with no tool calls. Each full cycle is one **turn**.
5. **Return result** — a final `AssistantMessage`, then a `ResultMessage` carrying final text, token usage, cost, and session ID.

```

import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage

async def run_agent():
    async for message in query(
        prompt="Find and fix the bug causing test failures in the auth module",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Edit", "Bash", "Glob", "Grep"],
            setting_sources=["project"], # loads CLAUDE.md, skills, hooks
            max_turns=30, # prevent runaway sessions
            effort="high",
        ),
    ):
        if isinstance(message, ResultMessage):
            if message.subtype == "success":
                print(f"Done: {message.result}")
            else:
                print(f"Stopped: {message.subtype}") # error_max_turns, error_max_budget_usd, ...

asyncio.run(run_agent())

```

Cap runaway loops with `max_turns` (tool-use turns only) and/or `max_budget_usd` (spend-based cap; also covers subagent spend). When either limit hits, the SDK returns a `ResultMessage` with an `error_max_turns` / `error_max_budget_usd` subtype rather than raising immediately mid-loop.

Custom agent loops and harnesses

You don't have to use the SDK's built-in loop wholesale — the `ClaudeAgentOptions` / `Options` surface (tool allow/deny lists, `permission_mode`, `effort`, `max_turns`, hooks) is designed to be composed into your own harness. A **harness** is the surrounding application logic that decides what prompt to send, which messages to surface to a user, how to handle a hit `max_turns` / `max_budget_usd` limit (e.g., resume the session with a higher limit), and how to persist/resume sessions across requests.

Managed agent deployment models: three wiring paths, ordered by how much infrastructure you hand off

The agent **loop itself doesn't change** — evaluate, call tools, feed results back, repeat — across all three paths. What changes is who runs it:

	Raw Messages API loop	Agent SDK (self-hosted)	Managed Agents (Anthropic-hosted)
Who runs the loop	Your code, every iteration — you read <code>tool_use</code> blocks, execute, append results yourself	The SDK, inside your own process — you still execute the tools it calls	Anthropic runs the loop <i>and</i> the sandbox — you send user events, stream results back over SSE
What you own	Everything: loop, tool execution, context management, retries, exit conditions	Tool execution and the surrounding application; the SDK provides loop structure, context management, tool registration	The application layer and the agent definition (model/prompt/tools/MCP/skills, referenced by ID)
Choose it when	Full control needed, or constraints a library doesn't accommodate	You want Claude Code's loop/context/tool scaffolding without rebuilding it, running in your own Python/TypeScript environment	Long-running execution (minutes-to-hours), you want a managed sandbox, or you'd rather not build the loop/sandbox/execution layer at all

	Raw Messages API loop	Agent SDK (self-hosted)	Managed Agents (Anthropic-hosted)
Watch for	Every SDK convenience (context management, parallel tool handling) becomes code you write and test yourself	Whether <code>settingSources / setting_sources</code> is set explicitly — do not rely on a default	Sessions are stateful and stored server-side — currently not eligible for Zero Data Retention or a HIPAA BAA . Public beta, requires the <code>managed-agents-2026-04-01</code> header.

The governing constraint that overrides convenience: if a workload carries PHI or falls under a ZDR requirement, Managed Agents is ruled out regardless of operational fit — route to the Agent SDK or a raw loop on a covered configuration instead. A common progression is prototyping on the Agent SDK locally, then moving to Managed Agents for production — expect a **re-expression step** (code/filesystem config → a versioned API resource), not a direct export.

A middle ground also exists at the self-hosted end: **self-hosted sandboxes** keep orchestration on Anthropic's side but move tool *execution* into infrastructure you control, so the agent's code, filesystem, and network egress never leave your environment while you still avoid running the orchestration loop yourself.

Human-in-the-loop: where the gate belongs, by worst-case cost

The design question that determines whether a checkpoint is needed: **what is the worst outcome if this step runs without a human check?**

Insertion point	Triggers on	Risk it addresses
Before a destructive tool call	A write, delete, or send operation is about to execute	High — irreversible actions, a wrong call can't be undone
After a planning step	A plan has been generated, execution is about to begin	Medium — an incorrect plan produces the wrong outcome even if every step executes correctly
On unexpected output	A tool result carries an error flag, an empty result, or an out-of-bounds value	Variable — catches failure modes retry logic alone won't resolve

Gotcha (real incident pattern): an agent with `read_file / write_file / validate_config` tools, tested exclusively in a scratch directory, correctly identified an out-of-range config parameter in a customer environment, corrected it, and passed `validate_config` — loop exited cleanly on the first iteration, exactly as designed. The corrected value was a rate limit a downstream application depended on; `validate_config` checked the schema's allowed range, not whether anything downstream relied on the old value. **The loop did exactly what it was built to do — the failure was that "validation passed on this file" and "write committed to a live customer environment" were treated as the same event, with no checkpoint between them.** The question that was never asked at design time: what's the worst outcome if `write_file` runs against production with no human check? If a tool can take an irreversible action in production, register that checkpoint requirement when scoping the tool surface — not after the first incident.

Agent memory: three scopes, and a gotcha about how they fail differently in production

Scope	What it means	Right for
In-context	All state lives in the active conversation, resent every turn	Single continuous sessions where the whole history is small enough to fit comfortably
External storage	State is written to a database at session end, read back at session start	The same user/task continuing across many separate, shorter sessions over time
Stateless	Each session starts fresh, no persistence	Fully independent jobs (e.g. a document formatter that transforms one file and terminates)

Gotcha (real incident pattern): an agent built for a support engineer worked flawlessly in development, run as single continuous 10–15 turn sessions — in-context memory held everything comfortably. In production, the *same total work* was spread across many shorter sessions over multiple days, with accumulated history

injected at the start of each one. By the fourth session, injected history alone exceeded 40K tokens *before the first tool call*; combined with the system prompt and tool schemas, over 45K tokens were consumed before any productive work began, and the agent started returning incomplete results — a symptom that initially looked like a tool-selection failure, not a memory-architecture one. **Development and production used different session shapes (one long session vs. many accumulating short ones), and in-context memory handles those two shapes very differently.** Measure expected state size per session (history + system prompt + tool schemas) against the context limit *before* choosing in-context as the default — the fix (externalize history, inject only the relevant subset at session start) is fast at design time and expensive as a production hotfix.

Hooks for deterministic actions

Hooks are callbacks that run **in your application process** (not inside the model's context window — they don't consume tokens) at defined points in the loop. They exist specifically because you can't rely on the model to reliably self-enforce a rule stated only in a prompt — a hook is a deterministic, code-level guarantee.

Hook	Fires	Typical use
PreToolUse	Before a tool executes	Validate inputs, block dangerous commands — can deny the call outright or modify its input
PostToolUse	After a tool returns	Audit outputs, trigger side effects
UserPromptSubmit	When a prompt is sent	Inject additional context
Stop	When the agent finishes	Validate the result, persist session state
SubagentStart / SubagentStop	Subagent spawns/completes	Track and aggregate parallel results
PreCompact	Before context compaction	Archive the full transcript before it's summarized away

A `PreToolUse` hook that rejects a call short-circuits the loop entirely — the tool never runs, and Claude receives the rejection as the tool result and typically tries a different approach. This is the primary mechanism the exam's Security and Safety domain (see `5_eval_debugging_security.md`) expects you to reach for when a prompt-level instruction ("please don't run destructive commands") isn't a strong enough guarantee.

Agent Patterns and Frameworks (4.9%)

Tool-use loops, memory, and context-window management

The tool-use loop *is* the agent loop described above — evaluate, call tools, feed results back, repeat until done. "Memory" in this context means what persists **across** sessions/turns beyond the live context window: session resumption (capturing a `session_id` to continue later with full prior context restored), and forking a session to branch into an alternate approach without mutating the original. Context-window management is covered in depth in `4_model_selection_prompting_context.md` (compaction, tool-output pruning, subagent isolation) — the short version for this domain is: **subagents are the primary architectural tool for keeping the main loop's context window from growing unbounded** on long or exploratory tasks.

Sub-agents as a scaling mechanism

Subagents work well for a handful of delegated tasks per turn, defined either **programmatically** (an `agents` dict passed to `query()`, each entry an `AgentDefinition` with `description`, `prompt`, `tools`, `model`) or as **filesystem-based** markdown files in `.claude/agents/`. For orchestration at the scale of dozens-to-hundreds

of coordinated agents — beyond what fits comfortably as turn-by-turn subagent delegation — the SDK exposes a separate `Workflow` tool that moves orchestration into a script executed outside the conversation's own context, rather than inside the manager agent's turn-by-turn loop.

Third-party agentic frameworks

The exam blueprint names three specifically — know what each optimizes for, not implementation detail:

- **LangGraph**: graph-based orchestration where agent steps are explicit nodes in a directed graph. The default choice for complex, *stateful* workflows where you want explicit control over every transition, at the cost of more upfront structure than a simple agent loop.
- **PydanticAI**: a Python-first framework built around type safety and schema validation without heavy orchestration machinery — the strongest choice when your priority is validated, typed inputs/outputs over multi-agent coordination.
- **Strands Agents**: AWS's open-source, model-driven agent framework, deeply integrated with Amazon Bedrock — the natural choice inside an AWS-centric stack.

The common thread the exam is testing: the **Claude Agent SDK is a provider-native primitive** (the loop, tool execution, MCP integration, and prompt-caching all built in and optimized for Claude specifically), while LangGraph/PydanticAI/Strands are **independent, cross-provider frameworks** you'd reach for when you need patterns (explicit state graphs, strict typing, a specific cloud's native integration) that the provider-native SDK doesn't provide out of the box — not because the SDK is somehow less capable at the core agent loop.